

R-Control IP4 – UDP Legacy API

What is this?

A very simple UDP interface: you send a line of text, the device answers with a line of text. No connection setup, no login.

This interface exists for existing installations. For new projects prefer the [HTTP/JSON API](#) or [Modbus TCP](#).

Important: No authentication, no encryption. Use only on an isolated network.

Getting there in 2 minutes

Step 1: Enable it

Web interface → **Settings** → **Interfaces** → **UDP Legacy API** → tick, check the port (default **30303**), save.

Step 2: The commands

Command	Effect	Reply
OUT1 1	switch output 1 on	OK
OUT1 0	switch output 1 off	OK
OUT1 2	toggle output 1	OK
OUT1 ?	read the state	0 or 1
T ?	read the temperature	e.g. 21.40
OUT1 IMP 00:00:03 1	impulse ON for 3 seconds	OK

Outputs are **one-based** here: **OUT1** to **OUT4**.

Step 3: Try it

On Linux/macOS with **netcat**:

```
echo "OUT1 1" | nc -u -w1 192.168.0.3 30303
echo "T ?"    | nc -u -w1 192.168.0.3 30303
```

On Windows with PowerShell:

```
$client = [System.Net.Sockets.UdpClient]::new()
$data   = [Text.Encoding]::ASCII.GetBytes("OUT1 1")
$client.Send($data, $data.Length, "192.168.0.3", 30303)
```

From here on follows the **complete reference part** with all details.

This document describes the **UDP Legacy API** of the R-Control IP4 from an integration perspective. It targets customers/partners who want to integrate the interface into their own system (PLC/SCADA, building automation, middleware, test systems).

Status: Implementation in `components/udp_api_legacy` (firmware repo). The UDP Legacy API is intentionally **minimalistic** and **not authenticated**.

1. Overview

Purpose

- Simple UDP-based control/query of relay outputs.
- Query the last temperature measurement (NTC sensor), if available.

Characteristics

- Transport: UDP/IPv4
- Payload: ASCII text, line-based
- Stateless (per datagram), no sessions
- **No authentication / no encryption** (assumes LAN/isolated network)

2. Activation & Configuration

The interface is enabled via the device configuration.

2.1 Configuration Keys

- `config.interfaces.udp_api_legacy.enabled` (bool)
 - `true` = UDP interface enabled
 - `false` = disabled (factory default typically `false`)
- `config.interfaces.udp_api_legacy.udp_port` (int)
 - UDP port of the server

2.2 Default Port

- In the factory configuration file (example `spiffs/config.json`) `udp_port: 30303` is often set.
- If the key is **not** set, the firmware uses **port 12345** as a fallback.

Recommendation: Set the port **explicitly** in the configuration.

2.3 mDNS / Service Discovery

If mDNS is enabled, the device registers the service when UDP Legacy API is enabled:

- Service: `_udp_api_legacy._udp`
- Port: matches `config.interfaces.udp_api_legacy.udp_port`

In addition, the `_http._tcp` service may contain TXT records (`udp_api_legacy=true/false`, `udp_port=<port>`) to make port discovery easier for integrators.

3. Transport & Data Format

3.1 UDP Socket

- Protocol: UDP (datagrams)
- Bind: `0.0.0.0:<udp_port>` (INADDR_ANY)
- IPv4: yes
- IPv6: no (socket is created as AF_INET)

3.2 Encoding / Line Processing

- Character set: ASCII (printable characters), partially case-insensitive (see commands)
- One datagram can contain **one or multiple command lines**.
 - Line separator: `\n`
 - Optional `\r` at end of line is tolerated (CRLF supported).
- Leading/trailing whitespace is stripped.

3.3 Maximum Datagram Size

- Receive buffer: **192 bytes payload** per UDP datagram.
- Larger datagrams are truncated and/or lead to incomplete lines → avoid.

3.4 Response Behavior

- For **each** valid command line, the device sends **one** response (UDP datagram) to the source address.
- If a datagram contains multiple lines, multiple response datagrams are sent accordingly.

4. Addressing Relay Channels

Relays are addressed as channels `OUT<n>`:

- `<n>` is **1-based**: `OUT1`, `OUT2`, ...
- Valid range: `1 .. N`
 - `N` equals the number of `config.peripherals.switch_outputs[]`.
 - Fallback: If the configuration is not readable, `N=4` is assumed.

Invalid channels are answered with `ERROR: Invalid channel`.

5. Commands

General syntax

- Tokens are separated by one or more spaces.
- Unknown commands are rejected.

5.1 Query Temperature

Request

- `T ?`

Response

- Success: `<tempC>\n` (float with 2 decimal places, e.g. `22.50\n`)

Error cases

- `ERROR: Sensor not ready\n` – no valid measurement available yet
- `ERROR: Sensor fault\n` – sensor status indicates a fault

5.2 Query Relay State

Request

- `OUT<n> ?`

Response

- `0\n` – relay off
- `1\n` – relay on

5.3 Switch Relay (Set/Reset/Toggle)

Request

- `OUT<n> 1` – turn on
- `OUT<n> 0` – turn off
- `OUT<n> 2` – toggle

Response

- Success: `OK\n`

5.4 Trigger Impulse

Starts a time-limited impulse on the output.

Request

- `OUT<n> IMP HH:MM:SS 1` – impulse ON for duration
- `OUT<n> IMP HH:MM:SS 0` – impulse OFF for duration

Parameters

- Duration format: `HH:MM:SS`
 - `MM` and `SS` must be within `0..59`
 - Total time must be `> 0`

Response

- Success: `OK\n`

Error cases

- `ERROR: Missing parameter\n` – e.g. too few tokens

- **ERROR: Invalid duration**\n – invalid time format or **00:00:00**
- **ERROR: Invalid parameter**\n – last parameter is not **0/1**

6. Response Codes & Error Texts

The UDP Legacy API uses simple text responses.

6.1 Success Response

- **OK**\n

6.2 Data Response

- Temperature: e.g. **22.50**\n
- Relay state: **0**\n or **1**\n

6.3 Error Responses (from firmware)

Response	Typical trigger
ERROR: Unknown command \n	Unknown command, wrong token count, wrong format
ERROR: Invalid channel \n	missing OUT , channel not numeric, or outside 1..N
ERROR: Missing parameter \n	impulse command without all parameters
ERROR: Invalid duration \n	duration not HH:MM:SS , minutes/seconds >= 60, or 0
ERROR: Invalid parameter \n	impulse level is not 0 or 1
ERROR: Internal error \n	unexpected error while executing
ERROR: Sensor not ready \n	temperature query before a measurement is available
ERROR: Sensor fault \n	invalid sensor status

Note: All responses end with \n.

7. Integration Notes (Best Practices)

7.1 UDP Characteristics

UDP is connectionless. For robust integration:

- Use **timeouts** (e.g. 200–1000 ms, depending on the network)
- **Retry** when there is no response (e.g. 1–3 attempts)
- Clearly match responses to requests (e.g. send exactly one command per request)

7.2 Idempotency

- Idempotent (good for retries):
 - **OUT<n> 1**, **OUT<n> 0**, **OUT<n> ?**, **T ?**
- Not idempotent (retries may cause side effects):
 - **OUT<n> 2** (toggle)

- `OUT<n> IMP ...` (may trigger multiple times)

Recommendation: For critical applications, avoid toggle and instead query (`OUT<n> ?`) and then set `0/1` explicitly.

7.3 Sequencing / Multi-line Datagrams

Multiple lines per datagram are possible, but responses are sent as individual datagrams. Recommendation for integrators:

- Send **exactly one command** per UDP datagram (simplifies matching/retry logic).

8. Security Notes

This interface is **not authenticated** and can switch relays.

Recommended safeguards:

- Use only in **trusted networks** (OT/LAN segment)
- Restrict access via **firewall/VLAN** (only defined source IPs/hosts)
- Keep UDP Legacy API disabled in normal operation (`enabled=false`) and enable only when needed

9. Examples

9.1 Example (Python, platform-neutral)

Sends one command and waits for a response.

```
import socket

DEVICE_IP = "192.168.0.3"
PORT = 30303
TIMEOUT_S = 1.0

def udp_request(cmd: str) -> str:
    data = (cmd.strip() + "\n").encode("ascii")
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(TIMEOUT_S)
        s.sendto(data, (DEVICE_IP, PORT))
        resp, _ = s.recvfrom(512)
        return resp.decode("ascii", errors="replace")

print("OUT1 ? =>", udp_request("OUT1 ?"))
print("OUT1 1 =>", udp_request("OUT1 1"))
print("T ?   =>", udp_request("T ?"))
```

9.2 Example (Set relay explicitly instead of toggle)

1. Read state: `OUT2 ?`
2. If `0`, then set: `OUT2 1`

9.3 Example (Impulse ON for 10 seconds)

- `OUT3 IMP 00:00:10 1`

10. Compatibility & Changes

The UDP Legacy API is a legacy interface and may be extended or adjusted in future firmware versions.

For projects with long-term maintenance requirements, it is recommended to use the authenticated JSON REST API or the WebSocket gateway instead (see [docs/API.md](#)).